
Software Frameworks

Reuse and Higher level designs

Frameworks, Components and Connection-Oriented Programming

From reusable code to connected workflows

- Advanced Programming Techniques
- Frameworks · components · plugins · connectors · visual workflows
- With Java/Spring-style examples and n8n as a connection-oriented programming case study

connect

frameworks
components
plugins
connectors
n8n
AI

2026

Part I

Frameworks

- A framework is not just a library.
- A framework defines a structure and calls your code at well-defined extension points.

Library vs Framework

Who controls the execution?

Library

Your code calls reusable functions or classes

Framework

The framework controls the application lifecycle and calls your code

Library example

Collections, Gson, JDBC driver APIs, utility packages

Framework example

Spring, Jakarta EE, JUnit, Android, JavaFX

The key concept is inversion of control.

Inversion of Control

The framework owns the main loop

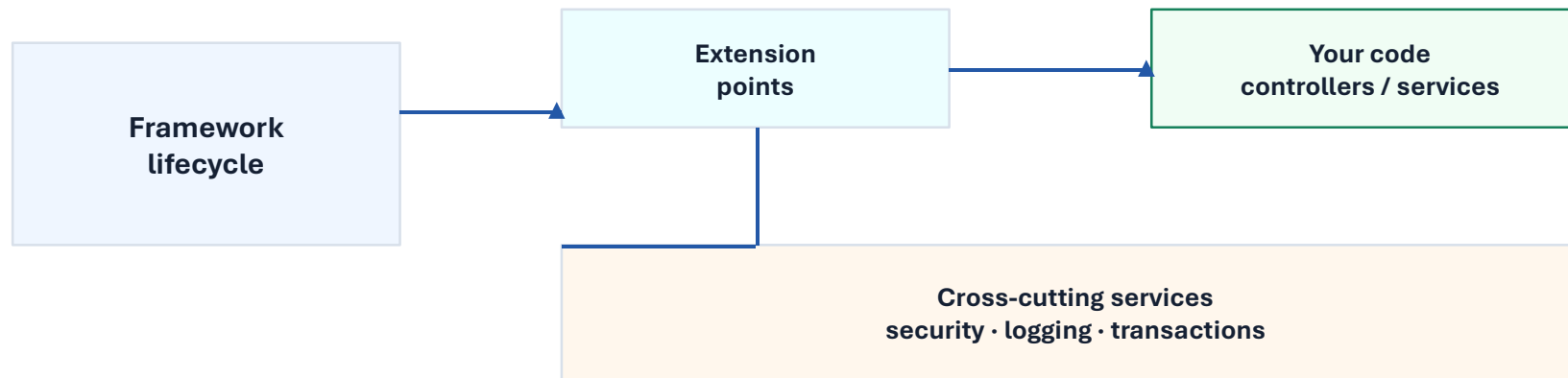
```
// Library style: your code is in control
String json = gson.toJson(student);

// Framework style: the framework calls your code
@RestController
class StudentController {
    @GetMapping("/students/{id}")
    Student find(@PathVariable String id) {
        return service.find(id);
    }
}
```

- Your code provides callbacks, handlers, services or controllers
- The framework creates objects and routes events or requests
- Annotations and configuration tell the framework how to connect parts

Frameworks Provide a Skeleton

Application-specific code fills the extension points



- Lifecycle: start, configure, execute, stop
- Extension points: controllers, services, repositories, filters, listeners
- Cross-cutting services: logging, transactions, security, configuration
- Conventions: project structure, naming, annotations, dependency injection

Dependency Injection Revisited

Frameworks connect objects for us

```
@Service
public class ReportService {
    private final ReportRepository repository;

    public ReportService(ReportRepository repository) {
        this.repository = repository;
    }

    public Report generate(String id) {
        return repository.findById(id);
    }
}
```

- The service does not create its repository
- The framework injects the dependency through the constructor
- The code depends on an abstraction or managed collaborator
- Testing is easier because dependencies can be replaced

Framework Annotations as Metadata

Annotations describe how code participates in the framework

```
@RestController
@RequestMapping("/reports")
class ReportController {
    private final ReportService service;

    @GetMapping("/{id}")
    Report get(@PathVariable String id) {
        return service.generate(id);
    }
}
```

- Annotations do not implement the business logic
- They provide metadata used by reflection or build-time processing
- The framework discovers, instantiates, routes and wires classes
- Reflection and annotations become practical, not just theoretical

Framework Hotspots and Frozen Spots

A useful way to understand framework design

Frozen spots

Parts fixed by the framework: lifecycle, routing, configuration model

Hotspots

Parts customized by developers: controllers, services, plugins, handlers

Good frameworks

Make common cases easy while preserving extension points

Bad use

Fighting the framework until the application becomes harder to maintain

Framework Benefits and Risks

Frameworks accelerate development, but also shape design

Benefits

Productivity, conventions, integration, security, testing support

Risks

Lock-in, hidden magic, steep learning curve, version upgrades

Design risk

Business logic can become coupled to framework details

Good practice

Keep domain logic separate from framework adapters

Part II

Component-Based Software Engineering

- Components package behaviour behind explicit interfaces.
- The goal is independent development, replacement and composition.

What Is a Component?

A reusable unit with explicit provided and required interfaces

- Encapsulates implementation details
- Offers services through provided interfaces
- Declares dependencies through required interfaces
- Can be deployed, tested, versioned or replaced independently
- Usually larger than a class and smaller than a complete system

A component is useful when its boundary is clear.

Class vs Object vs Component vs Service

Different granularities of software construction

Class

A code-level type and implementation unit

Object

A runtime instance with identity and state

Component

A deployable or reusable unit with explicit interfaces

Service

A component exposed through a network or platform boundary

Component Interfaces

Provided and required interfaces define the contract



- Provided interface: what the component offers
- Required interface: what the component needs
- Composition connects required interfaces to provided interfaces
- Contracts matter more than implementation details

Java Interface as a Component Boundary

The simplest component idea is an explicit contract

```
public interface ReportExporter {  
    String format();  
    void export(Report report, OutputStream out);  
}  
  
public final class CsvReportExporter implements ReportExporter {  
    public String format() { return "csv"; }  
    public void export(Report report, OutputStream out) { ... }  
}
```

- The client depends on the interface
- Implementations can be replaced or added
- The boundary supports testing, plugins and framework integration

Plugin Architecture

Components discovered and loaded at runtime

```
public interface Plugin {  
    String name();  
    void execute(Context ctx);  
}  
  
ServiceLoader<Plugin> loader = ServiceLoader.load(Plugin.class);  
for (Plugin plugin : loader) {  
    registry.register(plugin.name(), plugin);  
}
```

- Plugins extend an application without modifying its core
- The core defines extension contracts
- The plugin implements the contract
- Discovery can use configuration, reflection or Java ServiceLoader

Components and Composition

Software can be assembled from smaller parts

```
ReportPipeline pipeline = new ReportPipeline(  
    new CsvImporter(),  
    new GradeNormalizer(),  
    new PdfExporter()  
);  
  
pipeline.run("students.csv", "report.pdf");
```

- Each component has one focused responsibility
- The pipeline coordinates their collaboration
- Composition makes variation explicit
- This prepares students for workflow-based systems

Component Quality

A component boundary should reduce, not hide, complexity

- High cohesion: the component has one clear purpose
- Low coupling: dependencies are explicit and stable
- Clear contract: inputs, outputs, errors and side effects are documented
- Replaceability: alternative implementations can be used
- Observability: failures can be traced to the component boundary

Part III

Connection-Oriented Programming

- Instead of writing all control flow as code, we connect nodes or components.
- The program becomes a graph: data moves through connections.

What Is Connection-Oriented Programming?

Programming by connecting independent processing units

- The main abstraction is a graph of connected nodes
- Nodes perform operations: trigger, transform, filter, call an API, store data
- Edges define data flow, event flow or control flow
- The user designs a workflow by composing existing capabilities
- It appears in visual programming, ETL tools, serverless workflows and automation platforms

The connection becomes a first-class design element.

Imperative Code vs Connected Workflow

Two ways to express the same automation

```
// Imperative style
Order order = receiveWebhook(request);
if (order.total() > 1000) {
    slack.sendApprovalRequest(order);
} else {
    crm.createTicket(order);
}
```

```
// Connection-oriented style
Webhook -> IF(total > 1000) -> Slack
                        L-> CRM
```

- Code expresses control flow with statements
- Workflows express control flow with nodes and connections
- Both need testing, error handling and good design

n8n as an Example

A visual workflow platform based on nodes and connections

- Workflows are composed of connected nodes
- Trigger nodes start a workflow and supply initial data
- Action nodes perform tasks: call APIs, transform data, send messages or store records
- Flow logic controls branching, merging, loops and sub-workflows
- This makes n8n a good teaching example for connection-oriented programming

Terminology follows official n8n workflow and node documentation.

n8n Workflow Anatomy

Trigger, action nodes and data movement



data items move through connected nodes

- The trigger creates the first data item
- Each node receives input and produces output
- Connections define the next step
- The workflow can be tested node by node

Example Workflow: Student Report Notification

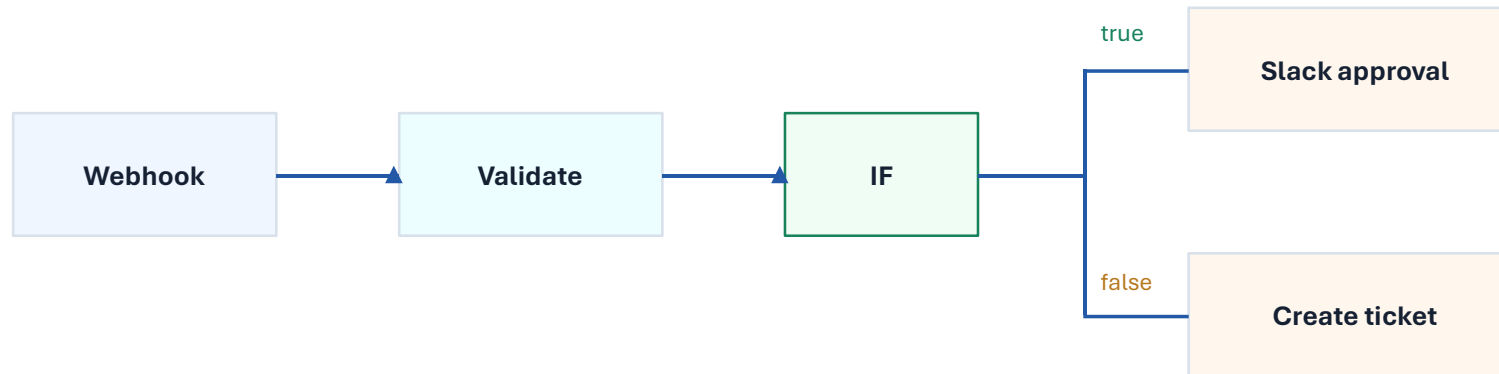
A classroom-friendly n8n scenario

```
Schedule Trigger
-> Read CSV from storage
-> Transform rows into student summaries
-> IF grade < 5
    -> Send email to tutor
    ELSE
    -> Store summary in database
-> Send completion message
```

- A trigger starts the process periodically
- Transformation nodes reshape the data
- Conditional nodes split the flow
- Actions integrate with external systems

Workflow as a Graph

The structure is visible and inspectable



- Branching is a graph operation
- The same data can follow different paths
- Errors, retries and compensation paths should also be designed
- The diagram is both implementation and documentation

Data Flow in n8n-Style Workflows

Nodes transform structured items

Input item:

```
{
  "student": "Ana",
  "grade": 4.2,
  "course": "TAP"
}
```

After transformation:

```
{
  "student": "Ana",
  "status": "needs review",
  "message": "Ana needs support in TAP"
}
```

- The workflow is only correct if each node receives the expected data shape
- Data mapping is a real programming skill
- Debugging means inspecting what each node received and produced

Connectors as Components

Workflow nodes package integration logic

Trigger connector

Starts the workflow from time, webhook, app event or queue

Action connector

Calls an external API, writes data, sends a message or performs a task

Transform node

Changes data shape, filters fields or computes derived values

Control node

Branches, merges, loops or calls another workflow

Connection-Oriented Design Questions

Good workflows still require architecture

- What is the source of truth for the data?
- Which node owns each transformation?
- What happens if an external API fails?
- Can the workflow be resumed without duplicating side effects?
- How are credentials, secrets and permissions managed?
- How will we test the workflow with realistic data?

Error Handling and Reliability

Visual workflows fail like any other distributed system

```
Webhook -> Validate input -> Call CRM API
                        |-> invalid -> Return 400
```

```
Call CRM API fails:
  retry with backoff
  log execution details
  notify operator
  avoid duplicate tickets
```

- Network calls can fail or timeout
- Retries can create duplicate side effects
- Validation should happen early
- Observability and execution history are part of the design

Frameworks vs Components vs Workflows

A useful comparison for students

Framework	Controls lifecycle and calls your extension code
Component	Encapsulates capability behind explicit interfaces
Workflow	Connects nodes/components into an executable graph
Connection-oriented programming	Focuses on the edges, data movement and composition rules

When to Use a Workflow Tool

And when ordinary code is better

Good fit

Integrations, automation, ETL, notifications, approvals, glue code

Risky fit

Complex algorithms, heavy domain logic, tight latency requirements

Hybrid design

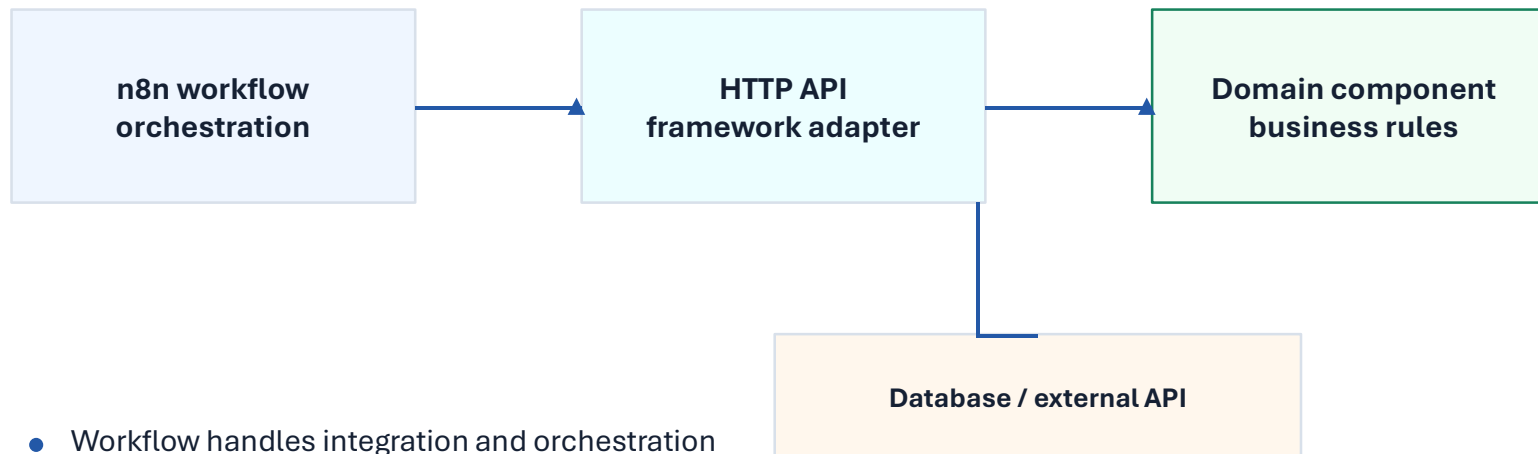
Use workflow for orchestration and code services for domain logic

Teaching point

Visual does not mean simple; it makes some complexity visible

Hybrid Architecture

Workflow orchestration plus code components



- Workflow handles integration and orchestration
- Services/components implement domain logic
- Frameworks expose APIs and manage infrastructure concerns
- The system combines all three ideas

Spec-Driven Workflows

Use specs to control visual and AI-assisted automation

Feature: notify tutors when a student needs review

Acceptance criteria:

- Given a grade below 5
- When the scheduled workflow runs
- Then the tutor receives exactly one notification
- And the execution is logged
- And invalid rows are reported separately

- Specifications clarify workflow intent
- Acceptance criteria define test cases
- Data contracts describe expected node inputs and outputs
- Specs help humans and AI agents modify workflows safely

AI-Assisted Workflow Development

AI can draft workflows, but humans must validate the graph

- AI can propose nodes, API calls, transformations and error paths
- The developer must check credentials, data contracts, privacy and side effects
- Tests should use realistic sample payloads
- Generated workflows need code review principles: clarity, minimality, observability

In connection-oriented programming, reviewing the connections is as important as reviewing the code.

Mini-Lab

Design a connection-oriented automation

- Choose a small academic workflow: grade report, assignment reminder or form processing
- Draw the nodes and connections before implementation
- Identify triggers, actions, transformations and error paths
- Define data contracts between nodes
- Write acceptance criteria and test payloads
- Implement it in n8n or as a small Java/Python pipeline

Key Takeaways

Modern software is assembled as much as it is written

- Frameworks invert control and provide application skeletons
- Components package reusable capabilities behind explicit contracts
- Connection-oriented programming makes composition and data flow central
- n8n is a practical example of workflows built from triggers, actions and flow logic
- Spec-driven and AI-assisted development require clear contracts, tests and reviewable connections

The advanced programmer understands code, components and connections.